

1. Field hiding in inheritance

Fields are resolved at compile time using reference type.

```
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;
}

Parent p = new Child();
System.out.println(p.x); // 10
```

2. Method overriding resolution

Instance methods are resolved at runtime using object type.

```
class Parent {
    void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child");
    }
}

Parent p = new Child();
p.show(); // Child
```

3. Parent reference to child object

Enables runtime polymorphism.

```
Parent p = new Child(); // Allowed
p.show();                // Child implementation
```

4. Child reference to parent object

Direct assignment is not allowed.

```
Parent p = new Parent();
// Child c = p;           // Compile error

Child c = (Child) p;      // Runtime ClassCastException
```

5. Overriding rules

Signature same, visibility cannot be reduced.

```
class Parent {  
    protected void test() {}  
}  
  
class Child extends Parent {  
    @Override  
    public void test() {}  
}
```

6. Covariant return type

Child may return subtype of parent's return type.

```
class A {}  
class B extends A {}  
  
class Parent {  
    A get() { return new A(); }  
}  
  
class Child extends Parent {  
    @Override  
    B get() { return new B(); }  
}
```

7. Visibility reduction

Reducing visibility causes compilation error.

```
class Parent {  
    public void run() {}  
}  
  
class Child extends Parent {  
    // protected void run() {} // ERROR  
}
```

8. Runtime polymorphism

Resolved using JVM virtual method table.

```
Parent p = new Child();  
p.show(); // JVM resolves dynamically
```

9. Compile vs runtime binding

Static/fields → compile time, instance methods → runtime.

```
p.staticMethod();    // Compile time  
p.instanceMethod();  // Runtime
```

10. LSP violation

Changing expected behavior breaks polymorphism.

```
class Parent {
    void pay() {}
}

class Child extends Parent {
    void pay() {
        throw new RuntimeException();
    }
}
```

11. final class

Prevents inheritance.

```
final class SecureService {}

// class Hack extends SecureService {} // ERROR
```

12. final method override

Final methods cannot be overridden.

```
class Parent {
    final void show() {}
}

class Child extends Parent {
    // void show() {} // ERROR
}
```

13. final calling overridden

Final method can call overridden methods.

```
class Parent {
    final void call() {
        show();
    }
    void show() {}
}

class Child extends Parent {
    void show() {
        System.out.println("Child");
    }
}
```

14. final dispatch

Final methods use static binding.

```
final void log() {}
```

15. final reference

Reference fixed, object mutable.

```
final List<String> list = new ArrayList<>();  
list.add("A"); // Allowed
```

16. final constructor

Constructors cannot be final.

```
// final Parent() {} // INVALID
```

17. abstract + final

Abstract class can have final methods.

```
abstract class A {  
    final void fixed() {}  
}
```

18. final + static

Completely locks method.

```
class Util {  
    static final void help() {}  
}
```

19. Immutable design

final fields prevent reassignment.

```
final class User {  
    private final int id;  
    User(int id) { this.id = id; }  
}
```

20. Security risk

Removing final enables malicious subclassing.

```
class BaseAuth {}  
class HackAuth extends BaseAuth {}
```

21. static override

Static methods are hidden, not overridden.

```
class Parent {
    static void show() {}
}

class Child extends Parent {
    static void show() {}
}
```

22. Hiding vs overriding

Static resolved by reference type.

```
Parent p = new Child();
p.show(); // Parent.show()
```

23. Static via object

Compiles but discouraged.

```
p.staticMethod();
```

24. Static on null

No NullPointerException.

```
Parent p = null;
p.staticMethod(); // Works
```

25. Static resolution

Resolved at compile time.

```
Parent.show();
```

26. Static accessing instance

Not allowed directly.

```
static void test() {
    // this.x; // ERROR
}
```

27. Static inheritance

Shared across hierarchy.

```
Child.staticVar++;
```

28. Static memory

Stored once per class in method area.

```
static int count;
```

29. Static polymorphism

Static methods don't support polymorphism.

```
Parent p = new Child();  
p.staticMethod(); // Parent version
```

30. Static bugs

Hidden methods cause unexpected behavior.

```
Debug.print(); // Wrong method
```

31. Abstract + interface

Partial implementation allowed.

```
abstract class A implements Runnable {}
```

32. Abstract extends abstract

Can override selectively.

```
abstract class B extends A {}
```

33. Abstract + interface

Allowed together.

```
class C extends A implements Runnable {}
```

34. Default vs abstract

Default has implementation.

```
interface I {  
    default void log() {}  
}
```

35. Multiple defaults

Class must override.

```
class X implements A, B {  
    public void log() {}  
}
```

36. Interface static

Not inherited.

```
InterfaceName.staticMethod();
```

37. Abstract static final

Used for utilities.

```
abstract class Util {  
    static final void help() {}  
}
```

38. Diamond problem

Resolved explicitly.

```
class X implements A, B {  
    public void log() {  
        A.super.log();  
    }  
}
```

39. Same method diff return

Not allowed.

```
// Compile-time error
```

40. Constructor order

Parent → Child order.

```
class A {  
    A() { System.out.println("A"); }  
}  
  
class B extends A {  
    B() { System.out.println("B"); }  
}
```